

Tune3 — Guia de execução dos experimentos

O que rodar, por que rodar, como rodar e como entregar

Projeto Tune3 — coordenação: Prof. Dionatan R. Schmidt
UNIPAMPA, Campus Alegrete — AI Horizon Labs

Versão de 21 de setembro de 2026 (revisão 2) — válida para o repositório a partir da tag
auditoria-2026-09-16

Resumo

Este guia é para quem vai *executar* os experimentos do Tune3. Ele não pressupõe que você conheça o método (para isso existe o documento *Fundamentos e arquitetura*), mas pressupõe que você vai seguir o protocolo à letra: os experimentos estão pré-registrados, e um resultado obtido fora do protocolo não pode ser usado. Cada teste tem um *propósito*, um *comando exato*, uma *saída esperada* e um *critério de aceitação*. Os resultados são entregues pelo próprio repositório, em arquivos JSON padronizados que o coordenador confere com um único comando.

Sumário

1	Regras do jogo (leia antes de instalar qualquer coisa)	3
2	Instalação passo a passo	3
2.1	Pré-requisitos	3
2.2	Clonar e criar o ambiente virtual	3
2.3	Instalar o PyTorch (com ou sem GPU) e o pacote	4
2.4	Baixar o DREBIN-215	4
3	Teste 0 — sanidade: o código faz o que a teoria diz?	5
3.1	Como ler a saída	5
4	Teste 1 — piloto rápido: o pipeline completo roda?	6
5	Os experimentos oficiais	7
5.1	H1 — in-distribution (20 sementes)	7
5.2	H2 — desbalanceamento (S1, 20 sementes)	7
5.3	H3 — zero-day por cluster (S2, 10 sementes $\times$ 5 folds)	7
5.4	Linha 1 — o que E1 e E2 significam nas saídas	8
5.5	E0 — reanálise (custo zero, depois do H3)	8
6	Quanto tempo isso leva	8
7	Como registrar e entregar os resultados	8
7.1	O que cada script grava	8
7.2	Entrega pelo git (é só isso)	9
7.3	O relatório curto	9
7.4	O que o coordenador vai fazer	9
8	Problemas comuns	9

9	Resumo em uma página	10
A	Campos do JSON que valem a pena conhecer	10

1 Regras do jogo (leia antes de instalar qualquer coisa)

Cinco regras não negociáveis

1. **Não altere parâmetros.** Sementes, épocas, orçamento, espaço de busca e limiares estão em PREREGISTRATION.md. Rodar “só para ver” com outros valores é permitido *apenas* com a flag `-tag exploratorio-<seu-nome>`, e o resultado não conta.
2. **Rode sempre de uma árvore limpa.** Antes de qualquer experimento: `git status` tem que estar vazio e `git pull` atualizado. Cada JSON grava o commit e se a árvore estava suja; resultado com `dirty = true` não é aceito.
3. **Nunca edite um JSON de resultado.** Se algo saiu errado, rode de novo com outra `-tag`. O histórico de tentativas é informação, não vergonha.
4. **Reporte o que aconteceu, não o que gostaria que tivesse acontecido.** Um experimento em que o Tune3 perde é um resultado científico; um em que o Tune3 “ganha” porque você trocou uma semente é fraude.
5. **Quando travar, pare e pergunte.** Erro que você não entende, tempo três vezes maior que o previsto, GPU sem memória: anote a mensagem completa e envie ao coordenador. Não “dê um jeito”.

O que você vai produzir. (i) Arquivos `results/<experimento>/<data>_<commit>_<seu-nome>.json`, commitados e enviados ao repositório; (ii) um relatório curto por rodada, a partir de `docs/TEMPLATE_RELATORIO_ALUNO.md`, salvo em `results/relatorios/`. Nada mais. Não é preciso mandar planilha, print de tela nem e-mail com números.

2 Instalação passo a passo

2.1 Pré-requisitos

- **Python 3.10 ou superior** (testado em 3.11 e 3.14). No Windows, instale pelo site python.org marcando “Add to PATH”. Verifique com `python -version`.
- **Git.** Verifique com `git -version`.
- **GPU NVIDIA** é recomendada para os experimentos oficiais (H1/H2/H3), mas *não* é necessária para o Teste 0 e o Teste 1. Verifique o driver com `nvidia-smi`.
- Cerca de 5 GB de disco (PyTorch com CUDA ocupa a maior parte).

2.2 Clonar e criar o ambiente virtual

Abra o PowerShell (Windows) ou um terminal (Linux/macOS) numa pasta de trabalho.

```
git clone https://github.com/schmidt-d1/tune3.git
cd tune3
python -m venv .venv
```

Ative o ambiente — este passo precisa ser repetido toda vez que abrir um terminal novo:

```
# Windows (PowerShell)
.\.venv\Scripts\Activate.ps1
# Linux / macOS
source .venv/bin/activate
```

O prompt passa a mostrar `(.venv)` no início. Se o PowerShell recusar por política de execução, rode uma vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e tente de novo.

Armadilha comum: dois ambientes

Se você usa PyCharm, ele pode criar um `.venv` próprio na pasta *acima* do repositório. Confirme sempre qual está ativo: no PowerShell, `$env:VIRTUAL_ENV` deve terminar em `\tune3\.venv`; no Linux, `echo $VIRTUAL_ENV`. O prompt `(.venv)` sozinho não garante nada.

2.3 Instalar o PyTorch (com ou sem GPU) e o pacote

```
python -m pip install --upgrade pip
# (a) COM GPU NVIDIA -- veja abaixo como escolher o índice:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu130
# (b) SEM GPU (ou notebook sem NVIDIA):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
# em ambos os casos, depois:
pip install -e ".[dev,vision]"
```

O cuXXX não é a versão do seu driver

O número no `-index-url` é o *build* do PyTorch, não o CUDA instalado na máquina. Ele precisa cobrir a *arquitetura* da sua GPU: placas RTX 50xx são Blackwell (`sm_120`) e exigem `cu128` ou superior — um build `cu124` instala sem erro e depois falha em toda operação. RTX 40xx aceita `cu121+`; RTX 30xx, `cu118+`. Além disso, cada índice só publica *wheels* para algumas versões de Python: com **Python 3.14** hoje só o `cu130` tem. Na dúvida, abra <https://download.pytorch.org/whl/cu130/torch/> e procure a linha com o seu `cp3XX` e sistema operacional.

Confirme — e note que `cuda.is_available()` pode dizer `True` com um build incompatível, então o teste tem que *calcular* alguma coisa:

```
python -c "import torch; print(torch.__version__, torch.version.cuda, torch.cuda.is_available()); print(torch.cuda.get_device_name(0), torch.cuda.get_device_capability(0)); x=torch.randn(1000,1000,device='cuda'); print('matmul ok :', float((x@x).sum()))"
python scripts/verify_setup.py
```

Se aparecer no `kernel image is available for execution on the device`, o build não cobre a sua GPU: volte ao alerta acima e instale de um índice mais novo. A segunda linha imprime `[ OK ]` para cada módulo do Tune3 e avisa o que falta. É normal aparecer `[FALTA] data/drebin215.csv` nesta etapa.

2.4 Baixar o DREBIN-215

O conjunto de dados não fica no repositório (é ignorado pelo git). Baixe o CSV do DREBIN-215 — figshare, *Android malware dataset for machine learning 2* (id 5854653), ou a cópia no Kaggle (“drebin 215 dataset”) — e salve como `data/drebin215.csv` dentro da pasta do repositório. Rode de novo:

```
python scripts/verify_setup.py
```

Saída esperada na seção de dados: `load_splits(): treino=(9622, 215) val=(2406, 215) teste=(3008, 215), features=215, frac_malware ≈ 0,37`. Se o número de atributos não for 215, o CSV está errado (coluna de classe mal identificada) — avise antes de continuar.

Quantas amostras o CSV deve ter — ponto em aberto

Os três números acima somam 15 036 aplicativos, que é o tamanho da distribuição mais comum do DREBIN-215. Documentos internos antigos do projeto mencionam 24 507 amostras, o que corresponde a *outra* distribuição do mesmo conjunto. As duas existem na internet. **Anote no relatório o total de linhas do seu CSV** (python -c "import pandas as pd; print(pd.read_csv('data/drebin215.csv').shape)"). Todos os alunos precisam usar o *mesmo* arquivo; o coordenador fixa qual é antes do início dos experimentos oficiais.

3 Teste 0 — sanidade: o código faz o que a teoria diz?

Por quê. Antes de gastar horas de GPU, precisamos saber que a instalação está íntegra e que cada componente matemático (estimador de curvatura, GSNR, CVaR, Cantelli, DDKF, EoS, estatística) se comporta como a teoria prevê *na sua máquina*. Este teste também é o que o coordenador vai olhar primeiro se algum resultado seu parecer estranho.

Passo 0.1 — suíte de testes (20–60 s):

```
pytest -q
```

Saída esperada: 89 passed (um *warning* sobre `torch.jit.script` em Python 3.14 é normal). Qualquer failed ou error: copie a saída inteira e pare.

Passo 0.2 — validação teoria × implementação, dados sintéticos (1–3 min, CPU):

```
python scripts/validate_theory.py --tag <seu-nome>
```

Passo 0.3 — a mesma validação com o DREBIN e a GPU (2–5 min):

```
python scripts/validate_theory.py --csv data/drebin215.csv --device cuda --tag <seu-nome>
```

3.1 Como ler a saída

O script imprime um veredito por item, no formato do projeto:

```
[PROVADO          ] V1 Hutchinson == Tr(H^2) exato (n=194 params)
    max|HVP - Hv|=1.5e-07; estimador(M=4000)=1.5858 vs exato=1.5872, z=-0.09
[VERIFICADO(escopo: este MLP)] V1b Hutchinson em model.train() (dropout=0.3) desvia do
    exato
    razao train/exato = 1.324 -> por isso o trial usa model.eval()
...
[VERIFICADO(escopo: t3, janela 10)] V5b Cantelli na janela REAL do trial
    taxa empirica=0.0087 (cota nominal 0.05; 24000 testes). Sem garantia formal
    com localizacao/escala estimadas -- e' verificacao empirica.
...
=== RESUMO: 13 itens, 0 REFUTADO(s), 2.5 min ===
[salvo] results/validation/20260921-171328_a1b2c3d_<seu-nome>.json
```

Item	O que testa	Aceitação
V1	o estimador de $\text{Tr}(H^2)$ contra a Hessiana exata	PROVADO; $ z < 4$
V1b	efeito do dropout na estimativa	VERIFICADO (razão $\neq 1$) — é informativo
V2	desvio do estimador cai como $1/\sqrt{M}$	razões entre 1,4 e 2,8
V3a/V3b	GSNR = definição; $\text{GSNR} \propto B$	PROVADO; razão em $[3, 20]$
V4	CVaR: duas fórmulas concordam; valor teórico	PROVADO
V5	Cantelli: alarme falso $\leq 5\%$ sob cauda pesada (janela 30)	VERIFICADO
V5b	o mesmo na janela que o <i>trial</i> usa de fato (30 épocas $\rightarrow$ janela 10)	VERIFICADO — é verificação empírica, não garantia formal
V6a	DDKF corrige na direção certa	VERIFICADO
V6b	engajamento do DDKF ao longo do tempo	EM ABERTO por desenho — não é erro
V7	EoS dispara no cenário certo e não no errado	PROVADO
V8	protocolo estatístico: $\text{FWER} \leq \alpha$, poder em $d_z = 0,8$	VERIFICADO
V9	ponta a ponta: 8 métodos rodam, Pareto, telemetria	VERIFICADO; anote a <i>duração</i>

Critério de aceitação do Teste 0: 0 REFUTADO(s). Se aparecer um REFUTADO, não siga: copie a linha inteira (item + evidência) para o relatório e avise o coordenador. O JSON em `results/validation/` é parte da entrega.

O que o V9 já está lhe dizendo

A linha do V9 termina com algo como `DDKF ativo em 0/85 epocas (regime A medio=1.00); EoS disparos=0`. Esse número vem de *uma* semente com orçamento mínimo (o V9 é um teste de integração, não uma medida do método), então não o leia como resultado — leia como o formato da telemetria de engajamento que o experimento E2 vai medir em escala, com 20 sementes e orçamento cheio. Anote no relatório.

4 Teste 1 — piloto rápido: o pipeline completo roda?

Por quê. O Teste 0 valida componentes. O Teste 1 roda o experimento H1 inteiro em miniatura (3 sementes, 20 épocas, orçamento reduzido) para (i) confirmar que os 8 métodos rodam de ponta a ponta com o DREBIN, (ii) medir o *tempo por semente* na sua máquina, do qual você vai extrapolar a duração dos experimentos oficiais, e (iii) você ver com os próprios olhos o formato da saída e do JSON. **Ele não produz significância** — com 3 sementes o menor p possível do Wilcoxon é 0,25; o script avisa isso.

```
python scripts/run_pilot.py --csv data/drebin215.csv --seeds 0 1 2 --epochs 20 --device cuda --tag <seu-nome>
```

(Sem GPU: `-device cpu`; leva mais tempo, mas funciona.)

Saída esperada (números ilustrativos):

```
===== RESULTADO DO PILOTO (CVaR no TESTE, menor=melhor) =====
tune3                : mediana=0.9813 IQR=...
tune3_bestcvar       : mediana=...
...
Seeds: 3 | p-minimo atingivel (Wilcoxon): 0.2500
```

```
AVISO: com este no de seeds e' IMPOSSIVEL atingir p<0.05.
tune3_bestcvar vs demais (pareado; regra pre-registrada = p_holm<0.05 E |d_z|>=0.30):
  vs bo_mono_cvar      : diff=..., d_z=..., IC95[...], p_holm=... [n.s.]
[salvo] results/pilot_h1/20260916-...json
```

Critério de aceitação: termina sem exceção; oito métodos listados; nenhum CVaR igual a 1000 (isso indicaria trials abortados em massa); JSON salvo. Anote a **duração** impressa (também está em `meta.duration_s` no JSON) — ela alimenta a tabela da Seção 6.

5 Os experimentos oficiais

Cada comando abaixo é uma linha só

Os comandos são longos e a caixa os quebra visualmente em mais de uma linha; essa quebra é **só da formatação**. Cole cada comando numa linha única — se o seu editor quebrou, junte antes de executar. Não use `^` para continuar linha (isso é do `cmd.exe` antigo, não do PowerShell) nem `\` (isso é do `bash`); no PowerShell o caractere de continuação seria a crase, que falha silenciosamente se sobrar qualquer espaço depois dela. Linha única funciona igual nos três terminais, e é por isso que os comandos estão assim.

Todos os comandos abaixo usam *todos os oito métodos* por padrão (Tune3 em três variantes, B4, RS, ASHA, SAM), o que cobre H1/H2/H3 e os controles da Linha 1 (E1, E2) de uma vez. Se o coordenador pedir um subconjunto, use `-methods` (ex.: `-methods tune3_bestcvar tune3_placebo_bestcvar bo_mono_cvar`).

5.1 H1 — in-distribution (20 sementes)

Pergunta. Em dados da mesma distribuição, o Tune3 tem vantagem? (Expectativa registrada: *não*; é um controle negativo.)

```
python scripts/run_pilot_parallel.py --csv data/drebin215.csv --seeds 0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 --epochs 100 --n-init 8 --n-iter 20 --device cuda --
workers 4 --tag <seu-nome>
```

`-workers 4` lança 4 processos que dividem as sementes e compartilham a GPU; se a memória da GPU estourar, use `-workers 2`.

5.2 H2 — desbalanceamento (S1, 20 sementes)

Pergunta. A vantagem em CVaR cresce quando o malware fica raro no treino (5%, 2%, 1%)? A predição falsificável é a *ordem* dos d_z nas três razões.

```
python scripts/run_s1_imbalance.py --csv data/drebin215.csv --seeds 0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19 --ratios 0.05 0.02 0.01 --epochs 100 --device cuda --
tag <seu-nome>
```

Além do CVaR, observe `test_auc_pr` e `test_fpr_at_95tpr`: sob forte desbalanceamento, F1 e MCC com limiar fixo degeneram e não devem ser interpretados.

5.3 H3 — zero-day por cluster (S2, 10 sementes × 5 folds)

Pergunta. Em malware de um “grupo” nunca visto no treino, o Tune3 best-CVaR vence RS, ASHA e SAM? É o cenário em que a tese prevê vantagem — e o único em que ela apareceu antes.

```
python scripts/run_s2_zeroday.py --csv data/drebin215.csv --seeds 0 1 2 3 4 5 6 7 8 9 --n
-clusters 5 --epochs 100 --device cuda --tag <seu-nome>
```

O script imprime duas análises. A **primária** usa *uma observação por semente* (média sobre os 5 folds). A “pooled” (folds $\times$ sementes) é **exploratória**: sementes do mesmo fold compartilham o conjunto de teste e não são independentes. Reporte a primária.

5.4 Linha 1 — o que E1 e E2 significam nas saídas

Você não precisa rodar nada a mais: os controles já estão nos comandos acima. Ao ler as tabelas, saiba o que cada linha responde:

- **bo_mono_cvar** (B4): a mesma otimização bayesiana e o mesmo trial interno do Tune3, mas otimizando *só* o CVaR. Se ele empata ou vence o Tune3, o segundo objetivo (curvatura) não está ajudando.
- **tune3_placebo_bestcvar**: o Tune3 com a curvatura trocada por *ruído*. Se empata com o Tune3 real, o ganho vem de ter dois objetivos, não da geometria.
- **tune3_noddkf_bestcvar**: o Tune3 com o controlador de learning rate desligado. Compare com **tune3_bestcvar** e olhe a telemetria no JSON (chave **_telemetry**: épocas com DDKF ativo e frações de regime).

5.5 E0 — reanálise (custo zero, depois do H3)

```
python scripts/analyze_e0.py --tag <seu-nome>
```

Lê os JSONs de **results/s2_zeroday/** e calcula, por fold e no total, a correlação parcial entre CVaR e $\log \text{Tr}(H^2)$ controlando $\log \eta$ e $\log \lambda_{wd}$. Funciona *apenas* sobre JSONs gerados pelo código corrigido (os antigos do Drive têm um defeito de avaliação e não servem).

6 Quanto tempo isso leva

O tempo depende da GPU. Use o Teste 1 para calibrar: seja T_1 a duração (em minutos) do piloto de 3 sementes $\times$ 20 épocas $\times$ orçamento 5 + 8. Os experimentos oficiais usam 100 épocas (com parada antecipada, tipicamente 30–60 efetivas) e orçamento 8 + 20, logo cada semente custa cerca de $2,2 \times (\text{orçamento}) \times 2\text{--}3 \times (\text{épocas}) \approx 5\text{--}7 \times$ uma semente do piloto. Estimativas:

Experimento	Sementes $\times$ cenários	Estimativa
H1 (run_pilot_parallel , 4 workers)	20	$\approx (20/3) \times 6 \times T_1/4$
H2 (S1)	20×3 razões	$\approx (60/3) \times 6 \times T_1$
H3 (S2)	10×5 folds	$\approx (50/3) \times 6 \times T_1$

Com $T_1 \approx 3$ min numa GPU de mesa recente: H1 ≈ 30 min, H2 ≈ 6 h, H3 ≈ 5 h. Planeje H2 e H3 para rodar à noite; não feche o terminal nem deixe o computador hibernar (no Windows: Configurações $\rightarrow$ Energia $\rightarrow$ “Nunca” suspender enquanto conectado).

7 Como registrar e entregar os resultados

7.1 O que cada script grava

Todo experimento grava **results/<experimento>/<AAAAAMDD-HHMMSS>_<commit>_<tag>.json** com dois blocos:

- **meta**: experimento, sua **-tag**, data, duração, *argumentos exatos*, commit do git e se a árvore estava suja, versão do Python/torch, GPU. É isso que torna o resultado auditável.

- **payload:** o resultado bruto (scores por método e semente, estatística pré-registrada, telemetria por trial).

Se você viu [AVISO] `arvore git SUJA` ao final, o resultado *não* é oficial: faça `git status`, resolva (normalmente é um arquivo que você editou sem querer), e rode de novo.

7.2 Entrega pelo git (é só isso)

```
git pull --rebase
git add results/
git commit -m "resultados: s2 zero-day, seeds 0-9 (<seu-nome>)"
git push
```

Os JSONs em `results/` são versionados de propósito (é a exceção no `.gitignore`). Não adicione nada fora de `results/` — se `git status` mostrar outros arquivos modificados, você mexeu em algo que não devia; avise.

7.3 O relatório curto

Copie `docs/TEMPLATE_RELATORIO_ALUNO.md` para `results/relatorios/<seu-nome>_<data>.md`, preencha e commite junto. Ele tem seis campos: identificação/máquina/commit; saída do Teste 0; tabela dos experimentos rodados (comando exato, duração, arquivo); o que você observou (fatos: medianas, d_z , `prereg_win`, telemetria); o que deu errado; dúvidas. Cole números, não adjetivos.

7.4 O que o coordenador vai fazer

```
python scripts/collect_results.py
```

Isso varre `results/` e monta uma tabela (terminal e `results/SUMMARY.md`) com: experimento, quem rodou, data, commit, árvore suja?, GPU, número de sementes, e, para cada comparação, d_z , p_{Holm} e `prereg_win`. É por essa tabela que os resultados de todos os alunos são conferidos lado a lado.

8 Problemas comuns

Sintoma	O que fazer
pytest: comando não reconhecido	O ambiente virtual não está ativo, ou o <code>pip install -e ". [dev,vision]"</code> não rodou nele. Ative e reinstale.
No module named 'tune3'	Você está fora da pasta do repositório ou o pacote não foi instalado (<code>pip install -e .</code>).
<code>torch.cuda.is_available() = False</code>	Instalou o torch de CPU (confirme: <code>torch.version.cuda</code> vem <code>None</code>). Desinstale (<code>pip uninstall -y torch torchvision</code>) e reinstale pelo índice certo da Seção 2.3. Confira o driver com <code>nvidia-smi</code> .
<code>no kernel image is available</code>	O build do torch não cobre a arquitetura da sua GPU (típico de RTX 50xx com <code>cu124</code>). Reinstale de um índice <code>cu128</code> ou superior.
CUDA out of memory	Reduza <code>-workers</code> (H1) para 2 ou 1. Não mude <code>batch_size</code> .
Um método mostra CVaR = 1000	Trials abortados (penalidade). Um ou dois por semente é normal em lr alto; se for a maioria, pare e reporte.
[AVISO] <code>arvore git SUJA</code>	Algum arquivo do repositório foi alterado. <code>git status</code> ; <code>git checkout - <arquivo></code> se foi sem querer; rode de novo.
Muito mais lento que a estimativa	Confira se está em <code>-device cuda</code> e se outra coisa usa a GPU (<code>nvidia-smi</code>).
<code>git push</code> rejeitado	<code>git pull --rebase</code> e tente de novo; resultados de outros alunos chegaram antes.

9 Resumo em uma página

1. Instalar: clonar, `.venv`, `torch` (CUDA ou CPU), `pip install -e "[dev,vision]", verify_setup.py`, baixar `data/drebin215.csv`.
2. Teste 0: `pytest -q` (89 passed) e `validate_theory.py` sintético e com DREBIN (0 REFUTADO).
3. Teste 1: `run_pilot.py -seeds 0 1 2 -epochs 20`; anotar a duração.
4. Oficiais, com a árvore limpa: H1 (`run_pilot_parallel`), H2 (`run_s1_imbalance`), H3 (`run_s2_zeroday`); por fim, E0 (`analyze_e0`).
5. Entregar: `git add results/ && git commit && git push` + relatório curto em `results/relatorios/`.

A Campos do JSON que valem a pena conhecer

Caminho no JSON	Significado
<code>meta.git.commit_short</code> , <code>meta.git.dirty</code>	commit usado; <code>true</code> = resultado não oficial
<code>meta.args</code>	argumentos exatos da linha de comando
<code>payload.scores[m]</code> (piloto)	CVaR de teste por semente, para o método <code>m</code>
<code>payload.stats.comparisons[b]</code>	<code>dz</code> , <code>dz_ci95</code> (BCa), <code>p_raw</code> , <code>p_holm</code> , <code>practical</code> , <code>pre-reg_win</code>
<code>payload.stats.underpowered</code>	<code>true</code> se o n° de sementes não permite $p < 0,05$
<code>payload.primary_per_seed</code> (S2)	análise primária por semente
<code>payload.results_by_fold[f]</code> <code>.per_seed[i]._telemetry.tune3</code>	lista de trials do Tune3: <code>hparams</code> , <code>cvar</code> , <code>curvature</code> , <code>n_ddkf_active_epochs</code> , <code>regime_fractions</code> , <code>n_eos_triggers</code>
<code>payload.report</code> (validation)	vereditos V1–V9 com as evidências numéricas